Codo Learn – Software Requirements Specification (SRS)

Version: 0.1 (Draft) | **Date:** October 25, 2026 | **Repository:** github.com/osamasmsmsm666-png/codo-learn

> **Basis and assumptions.** The repository is an early-stage Next.js project (13 commits). Its README is still the default template, so the product purpose is inferred from the name ("Codo Learn") and the stack. Items marked **[Assumption]** should be confirmed or replaced by the project owner.

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of **Codo Learn**, a web-based platform that helps people learn programming through structured lessons and interactive practice. It is the reference for developers, designers, testers and stakeholders.

1.2 Scope

Codo Learn is a responsive web application. Learners browse courses, take lessons, practice coding exercises and track progress. Administrators manage content. The first release (MVP) covers a content catalog, lesson viewer, exercises with feedback, progress tracking and user accounts.

Out of scope for MVP: native mobile apps, live video classes, payments, a full multi-language code execution cloud, and a social network.

1.3 Definitions

Term	Meaning
Learner	End user who studies on the platform
Admin	Staff user who manages courses and users
Course	Ordered collection of modules
Module	Group of lessons on one topic

Term	Meaning
Lesson	Single learning unit (text, code samples, optional exercise)
Exercise	Task the learner solves, with automated checking
XP / Streak	Gamification points and consecutive-day activity count

1.4 References

- Next.js documentation (App Router); AGENTS.md in the repository warns that the installed Next.js version has breaking changes, so developers must read `node_modules/next/dist/docs/` before coding.
- IEEE 29148 (requirements engineering); WCAG 2.2 AA.

2. Overall Description

2.1 Product Perspective

A standalone web application, deployed as a Next.js app (suitable for Vercel or any Node host). It talks to a backend/database layer **[Assumption: not yet present in the repo, to be added]**.

2.2 Current Technology Baseline (from the repository)

Layer	Technology
Framework	Next.js 16.4 (App Router: `app/` directory)
UI library	React 19
Language	TypeScript 5
Styling	Tailwind CSS 4
Animation / icons	Motion, lucide-react
Quality	ESLint 9 with eslint-config-next
Not yet present	Database, authentication, code-execution engine, tests, CI

2.3 User Classes

Class	Description	Priority
Learner (beginner)	New to programming; needs guidance and motivation	Primary
Learner (intermediate)	Wants practice and challenges	Secondary
Admin / Content editor	Creates and maintains courses	Primary
Guest	Unregistered visitor browsing the catalog	Secondary

2.4 Operating Environment

Modern evergreen browsers (Chrome, Edge, Firefox, Safari – latest two versions) on desktop, tablet and phone. Server: Node.js LTS.

2.5 Constraints

- Must use the existing stack (Next.js, React, TypeScript, Tailwind).
- Must follow the Next.js version-specific conventions noted in AGENTS.md.
- User data handling must meet applicable privacy law (GDPR-style consent and deletion).

2.6 Assumptions and Dependencies

- A managed database and authentication provider will be selected (e.g., PostgreSQL with an auth service).
- Code execution for exercises is either sandboxed in the browser (e.g., JavaScript in a Web Worker) or via a third-party sandbox API.
- Initial content will cover JavaScript/TypeScript, HTML/CSS and Python basics **[Assumption]**.

3. System Features and Functional Requirements

Priority: **M** = Must (MVP), **S** = Should, **C** = Could.

3.1 Authentication and Accounts

ID	Requirement	Pri
FR-AUTH-1	The system shall let users register with email and password.	M
FR-AUTH-2	The system shall let users sign in and out securely and reset forgotten passwords by email.	M
FR-AUTH-3	The system shall support social sign-in (Google, GitHub).	S
FR-AUTH-4	The system shall let users edit their profile and delete their account and data.	M
FR-AUTH-5	The system shall enforce role-based access (Learner, Admin).	M

3.2 Course Catalog

ID	Requirement	Pri
FR-CAT-1	The system shall list available courses with title, description, level and estimated duration.	M
FR-CAT-2	Users shall be able to search and filter courses by language, level and topic.	S
FR-CAT-3	Guests shall be able to view the catalog and a course preview without an account.	M
FR-CAT-4	Learners shall be able to enroll in and leave a course.	M

3.3 Lesson Viewer

ID	Requirement	Pri
FR-LES-1	The system shall render lessons containing formatted text, images and syntax-highlighted code blocks.	M
FR-LES-2	The system shall provide next/previous navigation and a module outline sidebar.	M

ID	Requirement	Pri
FR-LES-3	The system shall let learners mark a lesson complete and resume where they left off.	M
FR-LES-4	Learners shall be able to bookmark lessons and take private notes.	C

3.4 Interactive Coding Exercises

ID	Requirement	Pri
FR-EX-1	The system shall provide an in-browser code editor with syntax highlighting.	M
FR-EX-2	The system shall run learner code in a sandbox and compare output against test cases.	M
FR-EX-3	The system shall show pass/fail results per test case with clear error messages.	M
FR-EX-4	The system shall offer progressive hints and a reveal-solution option after a configurable number of attempts.	S
FR-EX-5	The system shall enforce execution limits (time, memory, output size) to prevent abuse.	M
FR-EX-6	Quizzes (multiple choice) shall be supported as an alternative exercise type.	S

3.5 Progress and Gamification

ID	Requirement	Pri
FR-PRG-1	The system shall show per-course and overall progress percentages.	M
FR-PRG-2	The system shall award XP for completed lessons and exercises.	S
FR-PRG-3	The system shall track daily streaks and display them on the dashboard.	S
FR-PRG-4	The system shall issue badges for milestones and a completion certificate for finished courses.	C

ID	Requirement	Pri
FR-PRG-5	The system shall provide a learner dashboard with current courses, next recommended lesson and recent activity.	M

3.6 Administration and Content Management

ID	Requirement	Pri
FR-ADM-1	Admins shall create, edit, reorder, publish and unpublish courses, modules and lessons.	M
FR-ADM-2	Admins shall define exercises, starter code and test cases.	M
FR-ADM-3	Content shall be authored in Markdown/MDX with live preview.	S
FR-ADM-4	Admins shall view and manage users and see basic usage analytics (signups, completions, drop-off).	S

3.7 UI/UX and Localization

ID	Requirement	Pri
FR-UX-1	The UI shall be fully responsive from 360 px phones to large desktops.	M
FR-UX-2	The UI shall support light and dark themes.	S
FR-UX-3	Animations (Motion) shall be subtle and respect the user's reduced-motion setting.	M
FR-UX-4	The platform shall support English and Arabic with right-to-left layout **[Assumption: confirm languages]**.	C

4. External Interface Requirements

4.1 User Interface

Key screens: Landing page, Catalog, Course overview, Lesson/Exercise workspace (split view: instructions and editor/output), Dashboard, Profile/Settings, Admin console, Sign in/Register.

4.2 Software Interfaces

Interface	Purpose
Database (PostgreSQL or equivalent)	Users, courses, progress, submissions
Auth provider / OAuth	Sign-in and sessions
Code sandbox (Web Worker or external API)	Safe execution of learner code
Email service	Verification and password reset
Analytics (privacy-friendly)	Product metrics

4.3 Communication

All traffic over HTTPS. Next.js route handlers or server actions expose the application API; JSON over REST.

5. Non-Functional Requirements

5.1 Performance

- NFR-PERF-1: Largest Contentful Paint under 2.5 s on a mid-range phone over 4G for the landing page and lessons.
- NFR-PERF-2: Exercise run feedback within 3 s for typical submissions.
- NFR-PERF-3: Support 1,000 concurrent learners at MVP without degradation.

5.2 Security

- NFR-SEC-1: Passwords hashed with a modern algorithm (Argon2 or bcrypt); no plaintext storage.
- NFR-SEC-2: Learner code must never execute on the application server process; use isolated sandboxes.

- NFR-SEC-3: Protection against XSS, CSRF, injection and rate-limit abuse; secure, HTTP-only session cookies.
- NFR-SEC-4: Secrets only in environment variables, never committed.

5.3 Reliability and Availability

- Target 99.5% monthly availability; daily database backups; graceful error pages.

5.4 Usability and Accessibility

- Conform to WCAG 2.2 AA: keyboard navigation, screen-reader labels, sufficient contrast, visible focus.
- A new learner should start their first lesson within 3 clicks of landing.

5.5 Maintainability

- TypeScript strict mode; ESLint clean; component-based structure under `components/`; unit tests for core logic and end-to-end tests for key flows; CI running lint, type-check, test and build on every pull request.

5.6 Portability and SEO

- Server-side rendering/static generation for public pages; semantic HTML, metadata and sitemap for search engines.

5.7 Privacy and Compliance

- Collect minimum personal data, provide consent and data export/deletion, and publish a privacy policy and terms.

6. Data Requirements (Logical Model)

Entity	Key attributes
User	id, email, name, role, locale, created_at
Course	id, slug, title, level, language, status
Module	id, course_id, title, order
Lesson	id, module_id, title, content, order, type
Exercise	id, lesson_id, starter_code, tests, hints
Enrollment	user_id, course_id, enrolled_at

Entity	Key attributes
Progress	user_id, lesson_id, status, completed_at
Submission	id, user_id, exercise_id, code, result, created_at
Badge / UserBadge	id, name, criteria / user_id, badge_id, awarded_at

7. Acceptance Criteria (MVP)

1. A new visitor can register, enroll in a course, complete a lesson and see progress update.
2. A learner can write code in the editor, run it and receive correct pass/fail feedback within 3 s.
3. An admin can publish a new lesson with an exercise and see it live without a code deploy.
4. All pages pass an automated accessibility check with no critical issues and meet the performance targets.
5. Lint, type-check, tests and production build pass in CI.

8. Open Questions

1. Who is the target audience (age, language, prior experience)?
2. Which programming languages must the first release teach?
3. Is the product free, freemium or subscription-based?
4. Should content be built-in (files in the repo) or managed in a database CMS?
5. Is Arabic/RTL support required at launch?

9. Revision History

Version	Date	Description
0.1	2026-10-25	Initial draft inferred from repository structure and stack